Grammar Processing-Left Recursion Risk Detection

RA2311026050248- Bamini A(CSE-AIML D)

Problem Understanding

In compiler design, grammar rules define the structure of a language. Some grammar rules contain left recursion, where a non-terminal appears at the beginning of its own production. This creates problems for top-down parsers like recursive descent parsers, as they may enter infinite loops.

The goal of this problem is to process grammar rules stored as raw strings and identify whether they contain left recursion. Additionally, we compute a feature called Recursion_Depth, which indicates how many times the left-hand side symbol appears in the right-hand side of the rule. This helps in analyzing the complexity of recursion in grammar definitions.

Logic Used

The program reads multiple grammar rules as input and processes each rule individually. First, the left-hand side (LHS) is extracted as the first character of the rule. Then, the right-hand side (RHS) is identified by locating the "->" symbol and copying the remaining part of the string. To compute Recursion Depth, the program scans the RHS and counts how many times the LHS symbol appears. This gives a measure of how deeply the rule is recursive.To detect left recursion, the program checks whether the RHS starts with the same symbol as the LHS. Before checking, any leading spaces are ignored to ensure accurate comparison.Finally, the program prints the grammar rule along with the computed Recursion_Depth and a flag called Has_Left_Recursion, where:

1 indicates presence of left recursion

0 indicates absence of left recursion

Justification of Features

The Recursion_Depth feature is useful to quantify how frequently recursion occurs within a grammar rule. Higher values may indicate more complex recursive patterns. The Has Left_Recursion flag is critical because left-recursive grammars are not suitable for top-down parsing techniques. Detecting this condition early helps in transforming the grammar into a suitable form, improving parser efficiency and correctness. Thus, these features act as parser-readiness indicators, enabling better preprocessing of grammar rules before parsing.

Conclusion

This implementation demonstrates how raw grammar definitions can be converted into structured insights using simple string processing techniques in C. The approach helps in detecting problematic grammar patterns and supports the development of efficient parsing systems.